Table des matières

Exercices corrigés 04 — Bâtir l'étoile des ventes 1
Questions courtes 1
Questions pratiques 2
Diagnostics 3
Questions à développer 4
Mises en situation 4
Corrigé 5

Exercices corrigés 04 — Bâtir l'étoile des ventes

Comment travailler ces exercices. Répondez seul, puis vérifiez avec le corrigé. Les exercices marqués [labo] portent sur ce que vous venez de faire au labo ; les autres vérifient que le cours est clair dans votre tête. Tout vient de la Théorie 04 et du Labo 04.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. La vue et la table. Nommez les quatre choses qui séparent une vue de staging d'une table de la couche `MARTS`.
2. Trois fichiers. Dites ce que portent `packages.yml`, `package-lock.yml` et `dbt_packages/`, et lesquels se versionnent.
3. La fourchette. Expliquez ce que déclare `[">=1.0.0", "<2.0.0"]`, et ce qu'on refuse en l'écrivant ainsi.
4. [labo] Le paquet oublié. Dites ce que fait dbt quand `packages.yml` déclare un paquet que personne n'a installé.
5. Les deux macros. Nommez les deux macros de `dbt_utils` employées dans ce cours, et ce que chacune fabrique.
6. La clé compilée. Sur une colonne qui n'est jamais vide, dites à quoi `generate_surrogate_key` se réduit une fois compilée.
7. Trois propriétés. Nommez les trois propriétés de la clé de substitution fabriquée par hachage, et celle qui change la façon d'écrire un fait.
8. Le prix de la clé recalculée. Le fait ne joint pas les dimensions pour retrouver leurs clés. Dites ce qu'on perd, et où ce trou se rebouche.

© 2026 iAlexMG. Tous droits réservés.

9. L’ordre des colonnes. Expliquez pourquoi une clé fabriquée sur deux colonnes n’est pas la même selon l’ordre où on les donne.
10. La valeur vide. Dites ce que la macro met à la place d’une valeur vide, et la conséquence quand deux lignes ont la même clé naturelle vide.
11. `date_spine`. Dites ce que la macro produit, le nom de la colonne qu’elle rend, et le traitement qu’elle réserve à la borne de fin.
12. La dimension plus longue que les faits. `dim_date` va jusqu’au 31 mars 2026 alors que les ventes s’arrêtent au 31 décembre 2025. Donnez les deux raisons.
13. La clé de `dim_date`. Dites pourquoi elle n’est pas un hachage, contrairement aux deux autres dimensions.
14. Les mois en anglais. Dites ce que rendent `strftime(date, '%B')` et `dayname()` sous DuckDB, et la parade retenue par la formation.
15. Ce qu’une dimension ajoute. Le modèle de `dim_concessionnaire` est court. Dites ce qu’il ajoute à la vue de staging dont il part, et pourquoi il n’a pas à faire plus.
16. Le type 1. Dites ce qu’une dimension de type 1 garde, ce qu’elle perd, et le chiffre qui mesure la perte chez Boréal.
17. La couche du milieu. Dites ce qu’on range dans `INTERMEDIATE`, et la raison qui le justifie.
18. Un fait mince. Définissez l’expression, et dites ce qu’elle change pour qui relit le modèle.
19. `ephemeral`. Dites ce que dbt écrit dans la base, ce qu’il liste quand même, et ce qu’on ne peut plus faire avec un tel modèle.
20. La marge et son taux. Dites lequel des deux se stocke dans un fait, lequel ne s’y stocke jamais, et la règle qui tranche.

1. Trois familles. Nommez les trois familles de colonnes d’un fait, avec un exemple de chacune dans `fct_ventes`.
2. [labo] Le témoin. Dites ce qui fait de la valeur nette des en-têtes de commande un témoin valable pour contrôler le fait, et pourquoi la somme des postes n’en serait pas un.

Questions pratiques

P1. [labo] Le contrôle du grain. Écrivez la requête qui vérifie le grain de `fct_ventes`, donnez le résultat attendu, et dites ce qu’un écart signifierait.

P2. [labo] La réconciliation. Écrivez la requête qui compare la valeur nette du fait à celle des en-têtes de commande et rend l’écart. Donnez les trois valeurs attendues.

P3. Les jours sans vente. Écrivez la requête qui compte, sur les années 2022 à 2025, les jours de `dim_date` qu’aucune ligne de `fct_ventes` ne touche, en distinguant les fins de semaine. Donnez les résultats et dites ce qu’ils confirment.

P4. La marge par gamme. Écrivez la requête qui donne, par gamme, le nombre de lignes, la valeur nette et la marge. Donnez le résultat.

© 2026 iAlexMG. Tous droits réservés.

P5. Recalculer une clé. Écrivez la requête qui prouve que `cle_concessionnaire` est bien le `md5` du code du concessionnaire, sans passer par dbt. Donnez l'attendu.

P6. Voir ce que dbt envoie. Donnez la commande qui produit le SQL réellement exécuté pour `fct_ventes`, dites où le fichier atterrit, et nommez les deux mécanismes du cours qu'on y voit à l'œil nu.

P7. Le lignage du fait. Donnez la commande qui liste tout ce dont `fct_ventes` dépend, sources comprises, et le résultat attendu. Dites ce qui vous surprend dans cette liste.

P8. (avancé) Le libellé de la province. `dim_concessionnaire` porte le code de la province, `QC`, et non son nom. Le libellé vit dans `raw.erp_t005u`, une ligne par province et par langue. Écrivez la vue de staging qui le rend lisible, branchez-la sur la dimension, et donnez les contrôles que vous lancez ensuite.

Diagnostics

E1. La clé qui ne joint rien. Un collègue écrit le fait ainsi :

```
  select
      date_commande as cle_date,
      {{ dbt_utils.generate_surrogate_key(['code_concessionnaire', 'numero_commande']) }}
          as cle_concessionnaire,
      ...
```

`dbt run` passe, le fait se construit, ses 75 652 lignes sont là. Dites ce qui ne va pas, ce que rend une jointure du fait vers `dim_concessionnaire`, et pourquoi rien n'a signalé le problème.

E2. Le taux qui se moyenne. Un collègue ajoute une colonne `taux_marge` au fait, calculée ligne à ligne, en expliquant que « ça évitera de le recalculer partout ». Un rapport affiche ensuite la moyenne de cette colonne. Dites ce que le rapport annonce, ce qu'il aurait dû annoncer, et d'où vient l'écart.

E3. Le `left join` prudent. Un collègue remplace la jointure interne de `int_lignes_enrichies` par un `left join` vers les commandes, « pour ne perdre aucun poste ». Le fait garde exactement le même nombre de lignes, et tous les contrôles passent. Dites pourquoi, et ce qui arriverait le jour où un poste n'aurait pas d'en-tête.

E4. Le jour qui manque. Un collègue écrit `end_date="cast('2026-03-31' as date)"` dans `dim_date`, en se disant que la période finit le 31 mars. Dites ce qu'il obtient, et le genre de symptôme que ce défaut produit plus tard.

E5. Le modèle qui ne se reconstruit pas. Un collègue corrige le calcul de la marge dans `int_lignes_enrichies`, lance `dbt run --select int_lignes_enrichies`, lit « Finished running » sans erreur, puis constate que `fct_ventes` porte toujours l'ancienne marge. Dites ce qui s'est passé et la commande à lancer.

© 2026 iAlexMG. Tous droits réservés.

Questions à développer

A. Recalculer ou joindre. Pour retrouver la clé d'une dimension dans un fait, deux écoles : la recalculer par la même macro, ou joindre la dimension et lire sa clé. Défendez les deux, dites laquelle ce cours a retenue et pourquoi, et nommez la condition sans laquelle le choix retenu devient dangereux.

B. Ce qu'une livraison doit dire. Le `LIVRAISON.md` du labo se termine par une phrase sur ce que l'étoile ne contient pas encore. Expliquez pourquoi cette phrase vaut plus que les quatre lignes du tableau qui la précèdent, et ce qui arrive quand elle manque.

C. (avancé) Livrer une dimension de type 1. Chez Boréal, 35,5 pour cent de la valeur vendue avant le 1er juillet 2024 est attribuée à un palier que le concessionnaire n'avait pas à l'époque. On vous demande si l'étoile peut être livrée dans cet état. Répondez, en donnant les conditions que vous posez et ce que vous écrivez noir sur blanc.

Mises en situation

M1. Microbrasserie de Trois-Pistoles. La microbrasserie vend des fûts aux bars et des canettes aux épiceries. Son logiciel de facturation exporte deux fichiers, les factures et leurs lignes, avec un code de produit, un code de client et un montant. On vous demande la première étoile. Dites le grain que vous déclarez et pourquoi, les dimensions que vous montez en premier, et le témoin indépendant que vous cherchez avant de livrer quoi que ce soit.

M2. (avancé) Coopérative forestière de Chibougamau. La coopérative change de logiciel de gestion. Le nouveau système renumérote tous les membres : le membre `M-0042` devient `4711003`. On vous demande de reprendre l'étoile existante, dont le fait porte des clés de substitution fabriquées par hachage du numéro de membre. Dites ce qui casse, ce qui tient, et la marche à suivre pour que l'historique reste interrogeable.

M3. Traiteur de Baie-Saint-Paul. Le contrôleur vous dit que le chiffre d'affaires de votre étoile diffère de celui de la comptabilité, de 1,4 pour cent, et que l'écart n'est pas le même tous les mois. Personne ne sait d'où il vient. Décrivez votre démarche pour le trouver, dans l'ordre, et ce que vous ferez du résultat selon les deux issues possibles.

© 2026 iAlexMG. Tous droits réservés.

Corrigé

Comment lire ce corrigé. Il donne la réponse et le pourquoi, jamais une notion neuve : tout ce qu'il emploie a été enseigné dans la Théorie 04 ou pratiqué dans le Labo 04. Quand une réponse repose sur un geste du labo, le renvoi le nomme.

Questions courtes

1. La vue et la table. Qui la lit : vos propres modèles, contre les analystes, les rapports et les autres formations. Ce qui la structure : la source, une vue par table livrée, contre la conception et son grain déclaré. La matérialisation : des vues, contre des tables. Ce qui la fait changer : un changement de la source, contre une décision de modélisation. *Théorie : `Theorie_04_AE.md`, section 1.*

2. Trois fichiers. `packages.yml` déclare ce que le projet veut et dans quelle fourchette de versions. `package-lock.yml` garde la version exactement résolue, pour que deux postes installent la même. `dbt_packages/` reçoit le code téléchargé. Les deux premiers se versionnent, le troisième jamais : il se reconstruit par `dbt deps`.

3. La fourchette. Elle accepte toute version 1 du paquet et refuse la version 2. Une version majeure change les signatures des macros, et personne ne veut que cela arrive au milieu d'une nuit de chargement. Elle laisse en revanche entrer les corrections et les ajouts de la version 1.

4. Le paquet oublié. dbt s'arrête à la première commande, sur un message qui dit exactement quoi faire : « dbt expects 1 package(s) based on packages specified in packages.yml, but found only 0 package(s) installed in dbt_packages … Run "dbt deps" to install package dependencies ». C'est un bon message, et ils ne le sont pas tous. *Labo : `Labo_04_AE.md`, étape 1.*

5. Les deux macros. `dbt_utils.date_spine` fabrique une ligne par jour entre deux bornes. `dbt_utils.generate_surrogate_key` fabrique une clé de substitution par hachage d'une ou plusieurs colonnes.

6. La clé compilée. Au `md5` de la valeur. Le SQL compilé est `md5(cast(coalesce(cast(<colonne> as TEXT), '_dbt_utils_surrogate_key_null_') as TEXT))`, et sur une colonne jamais vide le `coalesce` ne sert à rien : il reste le `md5` que vous obtiendriez en l'écrivant à la main. *Théorie : `Theorie_04_AE.md`, section 3.*

7. Trois propriétés. Déterministe, la même clé naturelle donne toujours le même hachage ; uniforme, une chaîne de 32 caractères quelle que soit la forme de la source ; calculable sans la dimension. C'est la troisième qui change la façon d'écrire un fait, puisqu'elle retire une jointure par dimension.

8. Le prix de la clé recalculée. On perd la garantie que la ligne existe dans la dimension : le fait peut porter la clé d'un concessionnaire qu'aucune dimension ne décrit, et la jointure d'un rapport perdra ces lignes en silence. Le trou se rebouche au Cours 05, avec le test `relationships`, et c'est pour cela qu'il n'y est pas facultatif.

9. L'ordre des colonnes. Parce que la macro concatène les valeurs avant de hacher, et qu'un hachage ne connaît que la chaîne qu'on lui donne. `md5('AB')` vaut `b86fc6b051f63d73…` et `md5('BA')` vaut `5fc810cf62601df8…` : deux chaînes différentes, deux clés sans rapport. L'ordre s'écrit une fois et ne se rediscute plus.

© 2026 iAlexMG. Tous droits réservés.

10. La valeur vide. Un texte sentinelle, `_dbt_utils_surrogate_key_null_`. La conséquence est que deux lignes dont la clé naturelle est vide reçoivent la même clé de substitution, et se confondent donc en une seule entité. Une clé se fabrique sur des colonnes obligatoires. *Théorie : `Theorie_04_AE.md`, section 3.*

11. `date_spine`. Une ligne par jour entre deux bornes, dans une colonne nommée `date_day`. La borne de fin est exclue : `cast('2026-04-01' as date)` donne un dernier jour au 31 mars.

12. La dimension plus longue que les faits. D'abord parce qu'une dimension de dates couvre un calendrier et non des faits : les jours sans vente sont des jours où la réponse est zéro, et une table qui les porte sait le dire. Ensuite parce que les trois mois de 2026 arrivent au Cours 07, et qu'une dimension qui les attend déjà n'aura pas à être refaite ce jour-là. *Théorie : `Theorie_04_AE.md`, section 4.*

13. La clé de `dim_date`. Parce qu'une date est stable, unique, lisible et ne change jamais de valeur : elle se suffit comme clé. La hacher coûterait la lisibilité et le tri sans rien rapporter. L'usage veut souvent un entier de la forme `20250131`, qui rend le même service.

14. Les mois en anglais. « January » et « Monday ». DuckDB n'a pas de notion de langue et aucune option ne change cela. La parade est un décodage explicite par une liste indexée, fait une seule fois, à l'endroit unique qui fabrique `dim_date` : ainsi le défaut ne se corrige pas ensuite dans chaque rapport. *Labo : `Labo_04_AE.md`, étape 4.*

15. Ce qu'une dimension ajoute. Une clé de substitution, et les libellés qu'un analyste veut voir dans un `where`. Elle n'a pas à faire plus parce que le gros du travail est derrière : le Cours 03 a déjà renommé, typé, décodé le palier et tiré l'état courant. Un modèle de dimension court est le signe que la couche précédente a fait son métier.

16. Le type 1. Elle garde la vérité d'aujourd'hui et perd l'histoire, entièrement. Chez Boréal, les 49 concessionnaires dont le palier a changé le 1er juillet 2024 comptent pour 90 314 267,26 $ de ventes antérieures à cette date, soit 35,5 pour cent de la valeur vendue avant la révision : c'est la part attribuée à un palier que le concessionnaire n'avait pas. *Théorie : `Theorie_04_AE.md`, section 5.*

17. La couche du milieu. Le travail qui n'est ni du staging ni du fait : joindre les postes à leurs en-têtes, écarter les postes refusés, calculer la marge. La raison est unique et suffit : que le modèle du fait reste lisible.

18. Un fait mince. Un fait qui ne contient que des clés et des mesures, la jointure, le filtre et le calcul métier vivant ailleurs. Pour qui relit, la différence est celle entre douze lignes qui se lisent d'un coup d'œil et quarante où plus personne ne sait ce qui vient d'où.

19. `ephemeral`. dbt n'écrit rien dans la base : le SQL est injecté dans le modèle qui l'appelle, sous la forme d'une expression de table commune. `dbt ls` le liste quand même, parce qu'il est dans le graphe. Ce qu'on ne peut plus faire, c'est l'interroger : `intermediate.int_lignes_enrichies` rend une erreur de catalogue, le schéma n'existant pas. *Labo : `Labo_04_AE.md`, étape 7.*

20. La marge et son taux. La marge se stocke, c'est une soustraction, donc une mesure additive : la somme des marges de dix lignes est la marge de ces dix lignes. Le taux ne se stocke jamais. La règle tranche en une phrase : on garde le numérateur et le dénominateur, le ratio se calcule à la lecture.

21. Trois familles. Les clés vers les dimensions, `cle_date`, `cle_concessionnaire`, `cle_produit`. Les mesures, `quantite`, `valeur_nette`, `cout`, `marge`. Les dimensions dégénérées, `numero_commande` et `numero_poste`, des identifiants du monde réel qui n'ont aucun attribut à eux.

© 2026 iAlexMG. Tous droits réservés.

22. Le témoin. Parce qu'il ne vient pas du même chemin : l'ERP calcule lui-même, sur chaque en-tête, la somme de ses postes non refusés, tandis que votre fait part des postes. Les deux calculs se rencontrent sans jamais s'être croisés. Une somme des postes refaite par vous ne serait pas un témoin, ce serait le même calcul une seconde fois, et il tomberait juste même si la logique était fausse. *Théorie : `Theorie_04_AE.md`, section 8.*

Questions pratiques

P1. Le contrôle du grain.

```sql
select
    count(*) as lignes,
    count(distinct (numero_commande, numero_poste)) as couples_distincts
from marts.fct_ventes;
```

Attendu : 75 652 et 75 652. Un écart signifierait qu'une jointure a dupliqué des lignes, et que le fait ne dit plus ce que son grain annonce. C'est le premier contrôle à faire sur tout fait, avant même de regarder les totaux : un total calculé sur un grain faux n'a aucun sens. *Labo : `Labo_04_AE.md`, étape 9.*

P2. La réconciliation.

```sql
select
    (select sum(valeur_nette) from marts.fct_ventes) as fait,
    (select sum(valeur_nette) from staging.stg_commandes) as erp_entetes,
    (select sum(valeur_nette) from marts.fct_ventes)
        - (select sum(valeur_nette) from staging.stg_commandes) as ecart;
```

Attendu : 430 148 012,16 $, 430 148 012,16 $, et 0,00 $. Aucune tolérance : ce n'est pas un contrôle statistique, c'est une égalité. *Labo : `Labo_04_AE.md`, étape 9.*

P3. Les jours sans vente.

```sql
with vendus as (select distinct cle_date from marts.fct_ventes)
select
    count(*) as jours_de_la_periode,
    count(*) filter (where v.cle_date is not null) as jours_avec_vente,
    count(*) filter (where v.cle_date is null) as jours_sans_vente,
    count(*) filter (where v.cle_date is null and d.est_fin_de_semaine) as dont_fin_de_semaine
from marts.dim_date d
left join vendus v on v.cle_date = d.date_jour
where d.annee between 2022 and 2025;
```

Attendu : 1 461 jours, 1 012 avec vente, 449 sans, dont 418 de fin de semaine. Les 31 qui restent sont des jours de semaine sans commande, jours fériés compris.

© 2026 iAlexMG. Tous droits réservés.

Ce que cela confirme est double. Boréal ne vend pratiquement pas la fin de semaine, ce qu'aucune requête sur le seul fait n'aurait pu montrer, puisqu'un jour sans vente n'y existe pas. Et une dimension de dates n'est pas un luxe : c'est elle qui permet de compter des absences.

P4. La marge par gamme.

```sql
select
    p.gamme,
    count(*) as lignes,
    sum(f.valeur_nette) as valeur_nette,
    sum(f.marge) as marge
from marts.fct_ventes f
join marts.dim_produit p on p.cle_produit = f.cle_produit
group by 1
order by 3 desc;
```

Gamme	Lignes	Valeur nette	Marge
Motoneige	3 708	154 146 280,98 $	39 526 456,54 $
Hors-route	4 911	133 674 387,82 $	33 600 274,77 $
Motomarine	2 148	83 432 098,97 $	20 658 461,75 $
Pièces et accessoires	64 885	58 895 244,39 $	23 578 471,64 $

La dernière ligne mérite un regard : les pièces font 86 pour cent des lignes du fait pour 14 pour cent de la valeur, et rapportent pourtant plus de marge que les motomarines. C'est le genre de constat qu'une étoile rend visible en trois jointures, et que la `RAW` cachait derrière cinq.

P5. Recalculer une clé.

```sql
select count(*) as ecarts
from marts.dim_concessionnaire
where cle_concessionnaire <> md5(code_concessionnaire);
```

Attendu : 0. La macro n'a rien de magique sur une colonne non vide, et c'est une bonne nouvelle : le jour où un chiffre surprendra, vous pourrez recalculer une clé vous-même, sans dbt, pour savoir de quel côté regarder. *Labo : `Labo_04_AE.md`, étape 5.*

P6. Voir ce que dbt envoie.

```
dbt compile --select fct_ventes
```

Le fichier atterrit dans `target/compiled/boreal_ae/models/marts/fct_ventes.sql`. Deux mécanismes du cours s'y voient à l'œil nu. Le modèle `ephemeral` est devenu une expression de table commune nommée `__dbt__cte__int_lignes_enrichies`, collée en tête de la requête. Et l'appel de macro est devenu son `md5(cast(coalesce(…)))`, où se lit le traitement des valeurs vides. *Labo : `Labo_04_AE.md`, étape 8.*

© 2026 iAlexMG. Tous droits réservés.

P7. Le lignage du fait.

```
dbt ls --select +fct_ventes
```

Six nœuds : `fct_ventes`, `int_lignes_enrichies`, `stg_commandes`, `stg_postes`, puis les deux sources `raw.erp_vbak` et `raw.erp_vbap`.

Ce qui surprend est la présence du modèle intermédiaire, qui n'existe nulle part dans la base. Il est dans le graphe parce que `ref()` l'y a mis, et le graphe ne parle pas d'objets, il parle de dépendances. Ce qui surprend en second, c'est l'absence des trois dimensions : le fait ne les joint pas, il recalcule leurs clés, donc rien ne l'y relie. Le graphe dit la vérité, et la vérité est que l'étoile ne tient que par une convention de hachage.

P8. (avancé) Le libellé de la province. Trois gestes, dans l'ordre.

La source d'abord, dans `_sources.yml` :

```
      - name: erp_t005u
        description: "Les provinces, avec leur libellé par langue."
```

Puis la vue de staging, dans `models/staging/stg_regions.sql` :

```
with courant as (

    select
        *,
        row_number() over (
            partition by land1, bland
            order by _lot desc
        ) as rang

    from {{ source('raw', 'erp_t005u') }}
    where spras = 'F'

)

select
    land1 as code_pays,
    bland as code_province,
    bezei as nom_province,
    _lot as lot_source

from courant
where rang = 1
```

Le filtre sur la langue est le point à ne pas manquer, et le Cours 02 l'a déjà montré sur les désignations d'articles : sans lui, la table rend deux lignes par province et la jointure double tout ce qu'elle touche. Mesuré : `dim_concessionnaire` passe de 156 à 312 lignes.

© 2026 iAlexMG. Tous droits réservés.

Puis la jointure, dans `dim_concessionnaire` :

```
select
    {{ dbt_utils.generate_surrogate_key(['c.code_concessionnaire']) }} as cle_concessionnaire,
    c.code_concessionnaire,
    c.nom_concessionnaire,
    c.ville,
    c.province,
    r.nom_province,
    c.palier,
    c.date_creation,
    c.est_supprime

from {{ ref('stg_concessionnaires') }} as c
left join {{ ref('stg_regions') }} as r
    on r.code_province = c.province
```

Les contrôles, et il en faut trois :

```
select count(*) as regions, count(distinct code_province) as provinces from staging.stg_regions;

select
    count(*) as lignes,
    count(distinct cle_concessionnaire) as cles,
    count(*) filter (where nom_province is null) as provinces_nulles
from marts.dim_concessionnaire;
```

Attendu : 10 régions pour 10 provinces distinctes ; puis 156 lignes, 156 clés, 0 province nulle. Le premier vérifie que le filtre de langue a fait son travail, le deuxième que la jointure n'a rien dupliqué, le troisième qu'aucun code de province n'est resté sans libellé. Le troisième contrôle est celui du Cours 02 sur le décodage : une jointure qui rate remplit de `NULL` sans rien signaler.

Un détail vaut d'être vu en passant : `BEZEI` est un `CHAR 20` dans SAP, et deux libellés arrivent donc coupés, « Terre-Neuve-et-Labra » et « Île-du-Prince-Édouar ». La source est ainsi, et ce n'est pas au `STAGING` de réécrire ce que le système source contient.

Enfin, le lignage a gagné une branche : `dbt ls --select +dim_concessionnaire` rend maintenant cinq nœuds au lieu de trois.

Théorie : `Theorie_04_AE.md`, section 5.

Diagnostics

© 2026 iAlexMG. Tous droits réservés.

E1. La clé qui ne joint rien. La clé du fait est fabriquée sur deux colonnes alors que celle de la dimension l'est sur une seule. Comme le hachage ne connaît que la chaîne qu'on lui donne, les deux clés n'ont aucun rapport.

La jointure du fait vers `dim_concessionnaire` rend zéro ligne. Mesuré : 75 652 lignes de fait, 0 jointe. Un rapport en jointure interne affiche donc un tableau vide, et un rapport en jointure externe affiche 430 148 012,16 $ attribués à un concessionnaire inconnu.

Rien n'a signalé le problème parce qu'il n'y avait rien à signaler du point de vue de dbt : les deux modèles sont valides, le fait a le bon nombre de lignes, les clés sont bien formées, et aucun lien ne relie le fait à la dimension dans le graphe, puisque le fait ne joint pas. C'est exactement le trou que le test `relationships` du Cours 05 vient combler. *Théorie : `Theorie_04_AE.md`, section 3.*

E2. Le taux qui se moyenne. Le rapport annonce 38,19 pour cent, là où le taux de marge réel de Boréal est de 27,28 pour cent. Onze points d'écart, sur un chiffre que personne ne recomptera.

L'écart vient de ce qu'une moyenne de taux donne le même poids à toutes les lignes. Les pièces et accessoires font 64 885 des 75 652 lignes du fait, avec un taux de marge de 40 pour cent, et elles écrasent la moyenne ; en valeur, elles ne pèsent que 14 pour cent. Le taux réel se calcule en divisant la somme des marges par la somme des valeurs nettes, jamais en moyennant des taux.

Le détail qui rend le cas vicieux : par gamme prise séparément, les deux calculs donnent presque la même chose, 24,52 contre 25,14 pour cent pour le hors-route par exemple. La distorsion n'apparaît que quand on mélange des lignes de tailles très différentes, c'est-à-dire dans le chiffre global, celui qu'on affiche en gros sur la première page. *Théorie : `Theorie_04_AE.md`, section 6.*

E3. Le `left join` prudent. Le nombre de lignes ne bouge pas parce que, chez Boréal, tout poste a son en-tête : mesuré, 0 poste actif sans commande. La jointure interne et la jointure externe rendent donc exactement le même résultat aujourd'hui.

Le jour où un poste n'aurait pas d'en-tête, la différence serait entière. Avec la jointure interne, la ligne disparaît du fait, et la réconciliation avec l'ERP le signale immédiatement. Avec le `left join`, la ligne entre dans le fait avec une date de commande vide et un code de concessionnaire vide ; la macro remplace alors le vide par son texte sentinelle, et la ligne reçoit la clé `f14cc5cdce0420f4…`, qui ne correspond à aucune ligne de la dimension. Le fait porte une ligne muette, et rien ne la signale.

La leçon n'est pas qu'un `left join` est mauvais, c'est qu'il faut savoir ce qu'on fait des lignes qu'il laisse passer. Ici, un poste sans en-tête est une anomalie de la source : on veut qu'elle fasse du bruit, pas qu'elle entre en silence.

E4. Le jour qui manque. Il obtient 1 550 jours au lieu de 1 551, le dernier étant le 30 mars 2026 : la borne de fin est exclue.

Le symptôme que ce défaut produit plus tard est le pire qui soit, parce qu'il est invisible pendant des mois. Tout fonctionne, tous les contrôles passent, et le jour où une vente tombera sur le dernier jour de la période, elle sortira de tout rapport qui joint le fait à la dimension des dates. Un total mensuel sera faux d'une journée, une fois par période, sans message d'erreur. Le contrôle qui l'attrape est celui du labo : compter les jours et regarder le dernier. *Labo : `Labo_04_AE.md`, étape 4.*

E5. Le modèle qui ne se reconstruit pas. Il ne s'est rien passé, littéralement. Un modèle `ephemeral` n'a aucun objet à créer, donc `dbt run --select int_lignes_enrichies` ne construit rien et ne rend même pas de ligne `PASS` : juste « Finished running » et un temps, ce qui ressemble à un succès.

© 2026 iAlexMG. Tous droits réservés.

Le SQL corrigé n'entrera dans la base qu'à la reconstruction du modèle qui l'appelle. La commande à lancer est `dbt run --select int_lignes_enrichies+`, avec le `+` qui prend tout l'aval, ou simplement `dbt run`. C'est la contrepartie de la matérialisation `ephemeral`, et elle se paie en attention.

Questions à développer

A. Recalculer ou joindre. Joindre la dimension pour lire sa clé a une vertu que rien d'autre ne donne : la jointure échoue à ramener une ligne quand le membre n'existe pas, et l'anomalie se voit tout de suite, au moment de la construction. Le fait est alors relié à ses dimensions dans le graphe de dbt, ce qui rend le lignage honnête, et l'ordre d'exécution garantit que les dimensions sont construites avant lui.

Recalculer la clé a les vertus inverses. Aucune jointure, donc un fait qui se construit vite et un modèle de douze lignes. Aucune dépendance, donc les dimensions et le fait peuvent se reconstruire dans n'importe quel ordre, ce qui compte le jour où l'étoile grossit. Et une propriété qui n'a l'air de rien : la clé ne dépend que de la clé naturelle, donc deux projets différents qui hachent la même colonne obtiennent la même clé.

Ce cours a retenu la seconde, parce que la première fait payer une jointure par dimension pour une vérification qui a un meilleur outil : un test, qui s'exécute quand on le décide, et dont l'échec nomme ce qui manque au lieu de le faire disparaître.

La condition sans laquelle ce choix devient dangereux est précisément là : il faut le test. Un fait qui recalcule ses clés sans test `relationships` est un fait dont personne ne vérifie jamais qu'il pointe quelque part. Le Cours 05 l'ajoute, et ce n'est pas une amélioration, c'est la moitié manquante du choix fait ici.

B. Ce qu'une livraison doit dire. Les quatre lignes du tableau se retrouvent en interrogeant la base : un compte de lignes se recompte, un grain se déduit du modèle, un nom de schéma se lit. Elles sont utiles, elles font gagner du temps, mais elles ne portent aucune information qui ne soit ailleurs.

La phrase sur ce qui manque, elle, ne se retrouve nulle part. Personne ne peut deviner, en regardant `dim_concessionnaire`, qu'elle écrase l'historique des paliers : la table a l'air complète, ses colonnes sont pleines, ses clés sont uniques. Un analyste qui l'utilise pour comparer les ventes par palier entre 2023 et 2025 obtiendra un résultat parfaitement formé et faux, et rien dans les données ne l'avertira.

Ce qui arrive quand la phrase manque est toujours la même chose, à retardement : quelqu'un construit un raisonnement sur une limite qu'il ignorait, le raisonnement remonte jusqu'à une décision, et la limite se découvre après. Le coût n'est pas la correction, qui est technique et courte, c'est la confiance perdue dans tout ce que l'équipe a livré, y compris ce qui était juste.

C. (avancé) Livrer une dimension de type 1. Oui, l'étoile se livre, et la question n'est pas de savoir si 35,5 pour cent est beaucoup : c'est de savoir à quoi elle va servir d'ici au Cours 05.

Une dimension de type 1 répond correctement à toute question qui porte sur l'état d'aujourd'hui : combien de concessionnaires Or avons-nous, où sont-ils, quel est leur chiffre d'affaires cumulé. Ce sont des questions légitimes et fréquentes, et elles n'ont pas à attendre. Ce qu'elle ne peut pas faire, c'est répondre à une question qui mêle le temps et l'attribut : combien avons-nous vendu à des concessionnaires Or en mai 2023.

Les conditions que je pose sont donc de cet ordre. La limite entre dans le `LIVRAISON.md`, en une phrase qui nomme le chiffre. Le tableau de bord qui croise le palier et une période antérieure à la révision attend, ou porte la mention. Et la date du correctif est prise, pas laissée à « plus tard », parce qu'une limite documentée sans échéance devient une limite permanente.

© 2026 iAlexMG. Tous droits réservés.

Ce que j'écris noir sur blanc, c'est la phrase qui permet à quelqu'un d'autre de décider sans moi : que `dim_concessionnaire` est de type 1, qu'elle attribue à toutes les ventes le palier d'aujourd'hui, que 49 concessionnaires ont changé de palier le 1er juillet 2024, et que cela touche 90 314 267,26 $ de ventes antérieures. Un chiffre dans une phrase vaut dix réunions.

Mises en situation

M1. Microbrasserie de Trois-Pistoles. Le grain que je déclare est la ligne de facture, une ligne par produit facturé. C'est le grain le plus fin que la source permet, et la règle de Kimball vaut ici comme chez Boréal : on prend le plus fin disponible, parce qu'on peut toujours agréger ensuite et jamais redescendre. Le grain « facture » interdirait d'analyser par produit, ce qui est précisément la question que la microbrasserie va poser en premier.

Les dimensions que je monte en premier sont la date et le produit. La date parce que tous les faits la partagent et qu'elle ne dépend d'aucune source, donc rien ne la bloque. Le produit parce que la distinction fût contre canette, et bar contre épicerie, est la question de métier du dossier : elle va se retrouver dans tous les rapports, et elle doit être une colonne lisible, pas un code.

Le témoin indépendant que je cherche est un total que le logiciel de facturation calcule lui-même et que je ne recalcule pas : le montant porté sur chaque facture, ou à défaut le total mensuel du rapport de ventes que le comptable reçoit déjà. La condition est qu'il ne vienne pas du même chemin que mon fait. Si le seul total disponible est la somme des lignes, je n'ai pas de témoin, et la première chose que je demande est l'accès à celui de la comptabilité.

M2. (avancé) Coopérative forestière de Chibougamau. Ce qui casse est complet : toutes les clés de substitution des membres changent. Une clé fabriquée par hachage ne dépend que de la clé naturelle, donc `md5('M-0042')` et `md5('4711003')` n'ont aucun rapport. Le fait historique pointe des clés que la dimension reconstruite ne contient plus, et toute jointure perd ses lignes.

Ce constat mérite d'être dit clairement, parce qu'il contredit un argument qu'on entend souvent : la clé de substitution protégerait d'un changement d'identifiant source. C'est vrai d'une clé indépendante, un compteur par exemple ; ce n'est pas vrai d'une clé qui hache la clé naturelle. Elle hérite de son instabilité.

Ce qui tient, en revanche, est tout ce que le fait porte en propre : ses mesures, ses dates, ses dimensions dégénérées. Aucune ligne n'est perdue, aucun montant n'est faux. Le problème est un problème de correspondance, pas de données.

La marche à suivre tient en trois temps. D'abord obtenir du fournisseur du nouveau logiciel la table de correspondance entre l'ancien et le nouveau numéro, et la faire entrer comme une source à part entière : sans elle, rien n'est possible, et c'est la première chose à réclamer avant la migration, pas après. Ensuite décider ce que porte la dimension : je garde les deux identifiants en colonnes, l'ancien et le nouveau, et je fabrique la clé sur celui qui ne changera plus. Enfin reconstruire le fait historique en traduisant ses clés par la correspondance, en vérifiant après coup que le total n'a pas bougé d'un cent et que le nombre de lignes est identique : c'est une renumérotation, pas une transformation.

Ce que je change pour l'avenir : la clé ne se fabrique plus sur un identifiant que le système source peut renuméroter. Si la coopérative n'a rien de stable à proposer, la clé devient un compteur tenu par l'entrepôt, et la correspondance vers la source devient une colonne comme une autre.

M3. Traiteur de Baie-Saint-Paul. Ma démarche suit l'ordre du coût, du moins cher au plus cher.

© 2026 iAlexMG. Tous droits réservés.

D’abord, je vérifie que les deux chiffres parlent de la même chose. Même période, même définition du chiffre d’affaires, taxes comprises ou non, avoirs et annulations compris ou non, même traitement des commandes annulées après facturation. Une différence de définition explique la majorité des écarts de cet ordre, et elle se trouve en une conversation.

Ensuite, je regarde la forme de l’écart. Un écart qui varie d’un mois à l’autre n’est pas un facteur constant, donc ce n’est ni une taxe ni une conversion : c’est un ensemble de lignes qui entrent d’un côté et pas de l’autre. Je descends alors d’un cran, mois par mois, puis semaine par semaine, pour trouver la période où l’écart est le plus net.

Ensuite seulement, je compare les deux populations sur cette période : les documents que mon fait porte et ceux que la comptabilité porte, par une jointure externe complète sur le numéro de document. Ce qui manque d’un côté, ce qui manque de l’autre, et ce qui existe des deux côtés avec un montant différent. Ces trois listes disent presque toujours la cause : un statut filtré, un type de document oublié, une date de rattachement différente.

Les deux issues, et ce que j’en fais. Si l’écart vient de mon étoile, je corrige, je rejoue la réconciliation, et j’écris ce qui s’était passé dans le `LIVRAISON.md`, parce que le prochain qui verra un écart de 1,4 pour cent doit savoir que celui-ci a déjà été cherché. Si l’écart vient d’une différence de définition légitime, les deux chiffres sont justes et c’est le plus fréquent : je ne change rien au calcul, j’écris la règle de rapprochement quelque part de visible, et je pose la question qui compte vraiment, celle de savoir lequel des deux chiffres est celui qu’on publie.

© 2026 iAlexMG. Tous droits réservés.